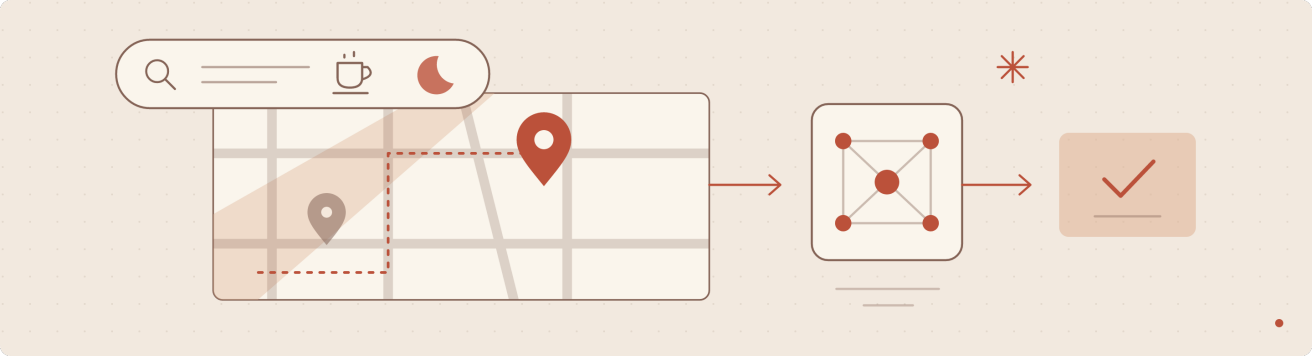
Агентная система для Яндекс.Карт

© 2026 Alexander Petrov. Written by Alexander Petrov.

Original publication: https://petrovortex.github.io/dls-agent-ru/



Intro

Недавно я закончил проект, которым занимался в рамках курса от Deep Learning School[1] (DLS). Нужно было построить LLM-агента для Яндекс.Карт, который помогает определить релевантность объекта на карте для запроса пользователя.

Задачка инженерная, и мне было жутко интересно: я изучал базу по агентам, пробовал различные архитектуры, занимался отладкой и, очевидно, много вайбкодил.

Мне есть что рассказать, и в своей статье я хочу поделиться полученным при выполнении проекта опытом. А именно, расскажу:

1. Про best practices разработки агентных систем;
2. Как я пришел к выбранной архитектуре;
3. Почему я не использовал фреймворки;
4. Как не разорился на тестировании (сделав около 4000 жирных запросов к LLM)

Подчеркиваю -- статья именно про мой опыт и путь выполнения проекта. Если вам интересен итоговый результат, переходите на гитхаб.

Я получил большое удовольствие от выполнения проекта и написания этой статьи, и в будущем планирую продолжать рассказывать о своих проектах. Недавно я устроился на новую работу, так что stay tuned :)

1. Как решать задачу?

Итак, дан датасет с колонками:

1. Запрос пользователя (user query),

2. Информация об объекте (point of interest info),
3. Истинная релевантность объекта для запроса: 0 - не релевантный, 0.1 - может быть релевантным, 1 - релевантный.

Требуется построить LLM-агента, который бы хорошо предсказывал релевантность объекта запросу пользователя на основе информации об этом объекте.



Несмотря на всю шумиху вокруг агентов, для меня этот проект стал первым местом, где я с ними столкнулся как разработчик, а не пользователь. Поэтому первым делом я принялся искать разные курсы и туториалы.

Мне повезло -- я нашел курс[2] от Эндрю Ына. Это крутой преподаватель из Стэнфорда, который умеет в любой теме выделять главное и понятно это объяснять.

В своем курсе Эндрю рассказывает:

1. Что такое агенты и зачем они нужны;
2. Про agentic workflows -- чему можно научить агентов;
3. Про best practices их разработки.

На самом деле, эти практики являются лучшими для разработки многих вещей, не только агентных систем, и тем ценнее является материал курса.

Разберемся по порядку. Если вы знакомы со всем этим, переходите к следующей главе, где я рассказываю про бейзлайн своего решения.

Что такое LLM-агенты?

Если грубо, то это одна или несколько LLM, которые **умеют что-то** делать помимо беседы в чатике. Но по этому поводу, оказывается, возникали дискуссии: является ли LLM, которая умеет, например, гуглить, агентом? Или нужно уметь что-то покруче?

Эндрю предлагает оставить эти споры, ведь важнее не иметь бинарную классификацию «агент / не агент», а понимать способности системы. В частности, **степень её автономности**:

1. **Низкая автономность (Hard-coded):** Вы как разработчик жестко прописываете последовательность шагов в коде (например: "Сначала поиск, потом саммари"). LLM просто выполняет задачи внутри этих шагов. Это надежно и предсказуемо.

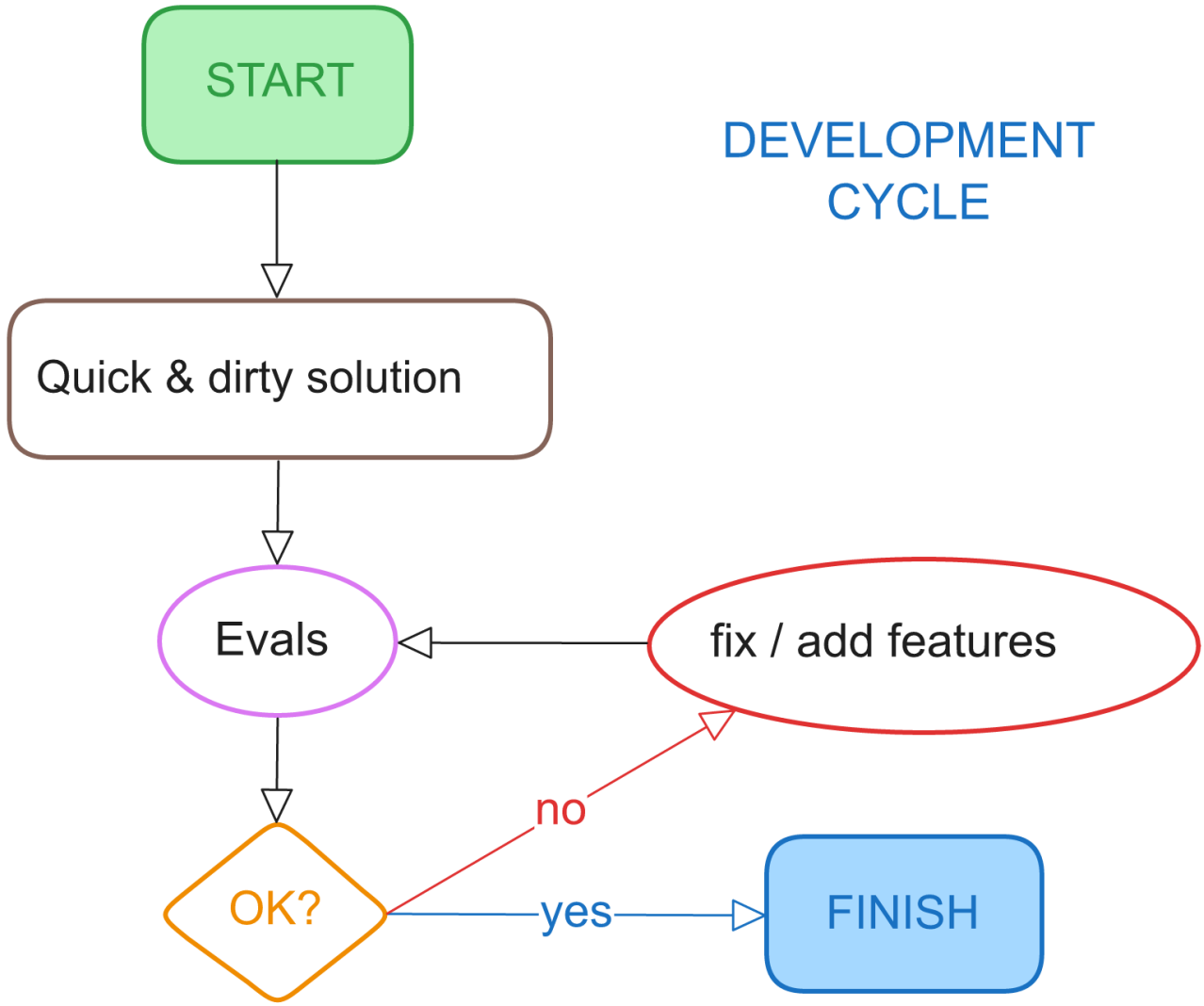
2. **Средняя автономность:** LLM может принимать локальные решения (например, "Нужно ли мне искать информацию или я знаю ответ сама?").
3. **Высокая автономность:** LLM сама определяет план действий, пишет и исполняет код, обрабатывает ошибки и итерируется до достижения цели.

Для решения разных задач можно построить разные агентные системы. Для каких-то достаточно одного запроса к LLM, для других необходимо устраивать сложный процесс взаимодействия нескольких LLM с разными компетенциями.

Как создавать агентов?

Заранее понять, какая агентная система будет оптимально хорошо решать задачу, почти невозможно. Поэтому Эндрю рекомендует:

1. Для начала сделать "быстрое и грязное решение";
2. Затем провести оценку качества работы и определить ошибки / слабые места;
3. Исправить ошибки / внести изменения;
4. Повторять пункты 2-3 до желаемого результата.

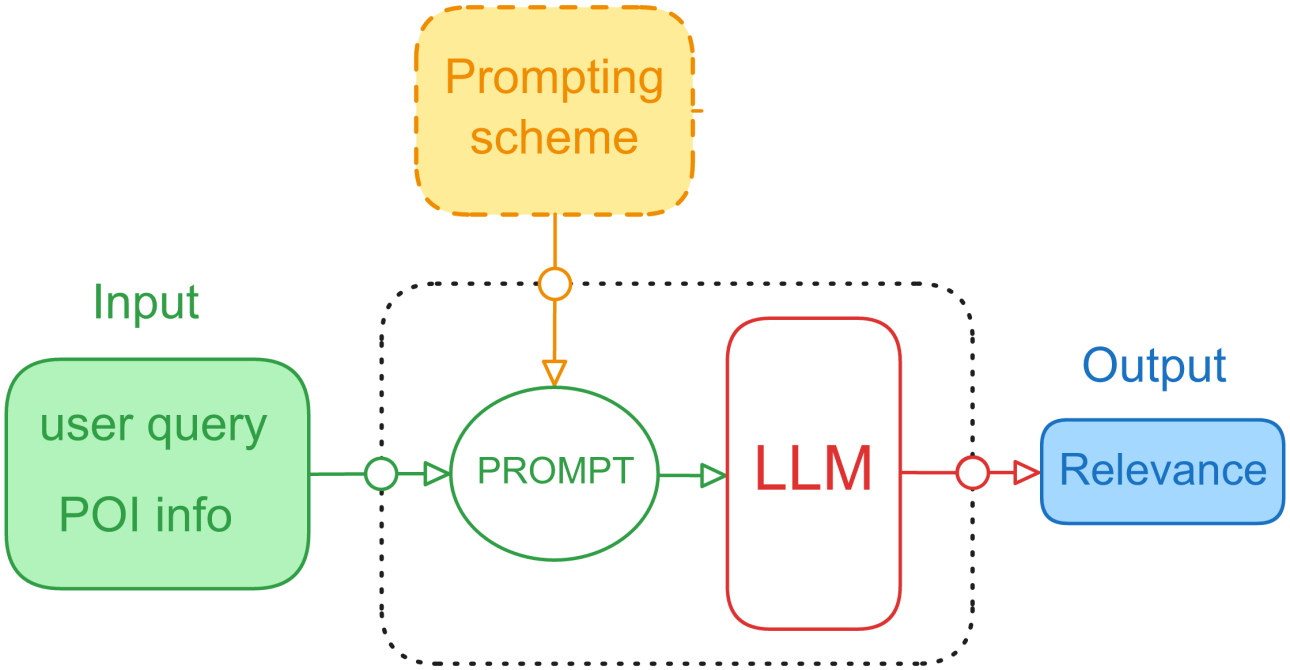


Дополняет этот цикл **важный совет**: сначала сделайте качественную работающую систему, а уже потом оптимизируйте её скорость и стоимость. Нет смысла оптимизировать систему, которая не решает поставленную задачу.

Подробнее почитать про workflows агентов и другие советы Эндрю вы можете в моем телеграм-канале, где я выложил свои конспекты. Либо пройдите его курс[2] самостоятельно -- рекомендую!

2. Быстрое и грязное решение

В качестве базового решения я выбрал **тривиального агента**: на вход в LLM подается информация об объекте и запросе, а на выходе ожидаются рассуждения модели (chain of thought), "на основе которых" модель определяет релевантность.



Дальше мне предстояло ответить на три вопроса:

1. Сколько я готов потратить на вызовы LLM?
2. Какие модели использовать?
3. Где проводить анализ ошибок?

Организаторы предупреждали, что эксперименты с агентами могут быть весьма затратными. Озвучивалась сумму **около 50-70$**. Учитывая мою природную невнимательность, я смело умножил эту сумму на два и понял, что не готов тратить столько на учебный проект.

Поэтому я решил использовать **бесплатные модели на openrouter.ai**. Проблему с ограниченным количеством запросов я решил, собрав кучу API-ключей с разных аккаунтов. Это в сумме давало мне до 400 бесплатных запросов в сутки, что вполне достаточно для проведения нескольких циклов.

Для **анализа ошибок** я нашел Opik[3] -- модный молодежный сервис для удобного контроля работы агентных систем. В нем можно отслеживать кастомные метрики, делать несколько экспериментов и сравнивать их между собой. Например, сравнивать работу агентов разной конфигурации: с разными LLM, базой знаний, веб-поиском, рефлексией и т.п. Все это мне пригодилось, но об этом попозже.

Пока что вернемся к базовому агенту. Я пробовал разные модели, которые на тот момент имели бесплатный тариф. Валидацию проводил на подвыборке из 50 объектов. Ответы оказались на уровне случайного гадания (accuracy=0.4), и я принялся за анализ логов работы агента.

3. Итеративное улучшение качества

Мне показалось логичным не усложнять систему на данном этапе, а попробовать добиться лучшего качества за счет **улучшения промпта**.

Улучшение промпта

Я посмотрел на ошибки агента и понял, что ему нужно дать некоторое общее представление о том, какие бывают объекты и запросы. Это поможет менее мощным моделям решить задачу лучше. Я придумал два способа, как это можно сделать:

1. Привести примеры с верным определением релевантности;
2. Сформулировать список правил, которых стоит придерживаться.

На самом деле существует третий способ, но его я реализовать не успел, поэтому расскажу о нем позже.

Для того чтобы агент понял логику разметки, нужно было предоставить ему как минимум три примера с разными значениями релевантности. Однако в рамках каждого класса (по релевантности) существуют нюансы (см. инструкции в промпте), которые стоит подсказать слабым моделей (10-100b параметров).

Если по каждому такому нюансу приводить примеры, то промпт сильно вырастет, и модель будет страдать уже от забывания контекста. Поэтому я выбрал второй способ и *вручную анализировал ошибки, чтобы сформулировать список правил*. Конечно, я прогонял свои правила через мощную LLM, чтобы она написала их в подходящем для LLM формате :)

С таким списком инструкций я смог добиться accuracy=0.55 (модель mimo-v2-flash), что было на 0.15 лучше базового решения.

Оптимизировав промпт, я перешёл к структурным изменениям агента. Начать решил с самого простого -- добавил **рефлексию**.

Рефлексия и веб-поиск

В самом простом случае рефлексия -- это повторный вызов LLM, в котором мы просим снова оценить релевантность, но, помимо информации об объекте и запросе, также передаем chain of thoughts из предыдущего вызова. В моих экспериментах такая реализация не дала заметного улучшения предсказаний, зато в два раза увеличила время их получения.

Почему улучшения не произошло? Я предполагал две причины:

1. Рефлексирующая модель слабая;
2. Недостаточно информации об объекте;

Эксперименты показали, что оба предположения верны. Действительно, если взять модель мощнее (в моем случае GLM-4.5-air), то она будет почти идеально определять релевантность объектов, *о которых информации достаточно*, а остальным присваивать метку 0.1.

Чтобы среди этих оставшихся выделить объекты с истинной релевантностью 0/1 необходимо дать агенту недостающую информацию. К примеру, если пользователь спрашивает дикси 24 часа, а в информации об объекте режима работы нет, то нужно пойти в поисковик и найти эту конкретную информацию. Это легко делается с помощью Tavily[4].

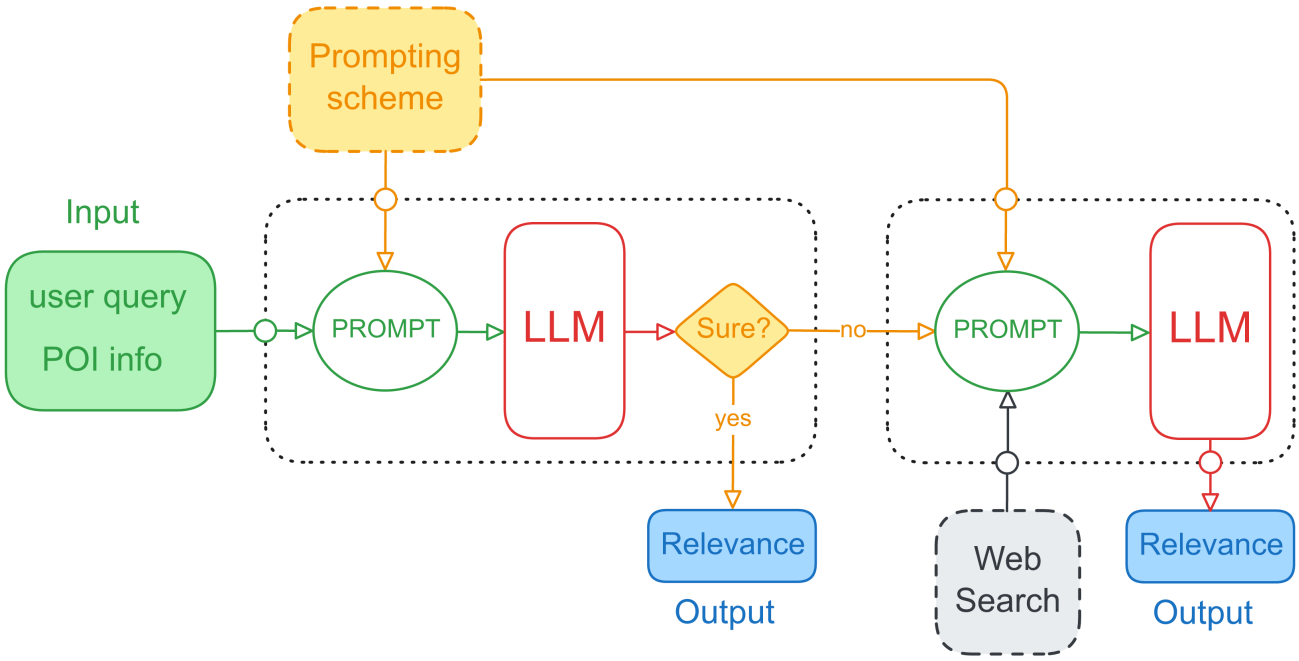
А много ли вообще объектов с недостатком информации? Оказалось, что их всего около 30%.

Это означает, что рефлексирующую модель можно вызывать только на таких объектах. Нужно лишь научиться с высокой точностью определять недостаток информации. Для этого нужно:

1. Выбрать достаточно умную модель (а значит, её можно оставить и на втором этапе!);
2. Сделать указание об этом в промпте.

Более того, можно вообще не передавать в рефлексирующую модель chain of thought из первой модели. Ведь они для данных объектов сводятся к тому, что не хватает определенной информации, а эту информацию мы получаем из поисковика и передаем второй модели.

Получается, что здесь нет явной рефлексии -- она заменена анализом результатов поискового запроса, который был сформулирован первой моделью.



Данная схема позволила мне добиться accuracy=0.75 для GLM-4.5-air, что было уже довольно хорошим результатом.

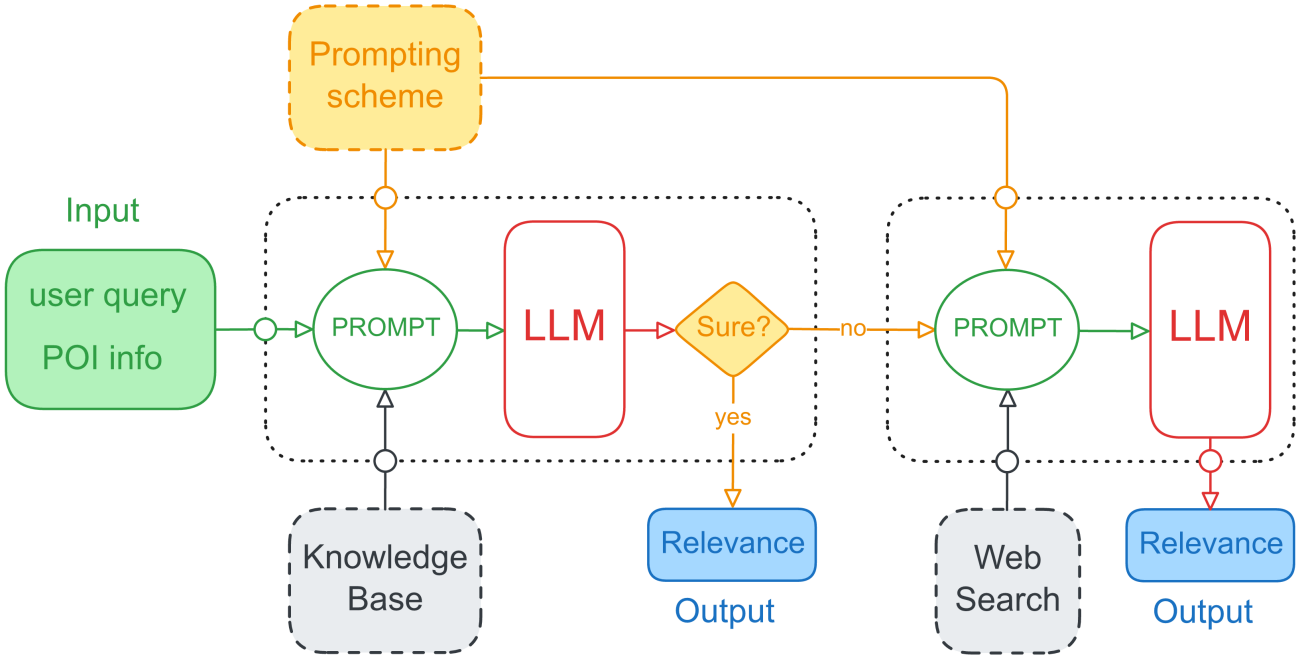
База знаний

Напомню, датасет проекта содержит пары (запрос пользователя, объект на карте), и изначально он был разделен на две части. На первой, тренировочной, нам разрешалось смотреть на ошибки агента, а вторая была чисто для тестирования точности агента (без анализа ошибок).

И у меня возник вопрос: а что, если в датасете есть одинаковые объекты? Тогда информация о релевантности запросов относительно этих объектов поможет определить их релевантность для других запросов!

Это оказалось правдой -- в датасете есть повторяющиеся объекты. Поэтому я разделил тренировочную часть датасета на две: из одной сделал векторную базу знаний, а на второй проводил валидацию.

Как это реализовано? На вход первой модели теперь подается (запрос, объект; 2 похожие пары (запрос, объект)). Похожие пары ищутся с помощью векторного поиска по базе знаний.



Внедрение БЗ позволило получить лучшее качество -- теперь агент на основе GLM-4.5-air выбивал accuracy=0.86.

Оптимизация затрат и скорости

Впервые получив accuracy=0.86 я уже знал, что "истинная релевантность" из датасета не всегда корректна. Это подтверждали и организаторы. Поэтому дальнейшее улучшение качества могло привести к подгону под кривую разметку. И я решил, что на данном этапе стоит заняться оптимизацией затрат и скорости.

После добавления примеров из базы знаний системный промпт заметно увеличился в размерах (несмотря на то, что примера всего два). И небольшие модели стали страдать от забывания контекста, а большие модели работали очень долго на примерах с длинным описанием объекта. Я задался вопросом: а можно ли подавать не все данные об объекте?

Действительно, если пользователь спрашивает про кафешки, работающие в час ночи, то для определения релевантности агенту не важны отзывы об уютности атмосферы и меню. Поэтому я реализовал векторный поиск релевантной информации из описания объекта. Помогло мне в этом то, что описание объекта имеет определенную структуру, и вычленять отдельные смысловые части (разбивать на чанки) оказалось легко.

Теперь на вход модели подается не вся информация об объекте, а только её участки, которые могут помочь оценить релевантность объекта для конкретного запроса.

Это позволило улучшить качество предсказаний для объектов с длинным описанием:

1. Большие модели стали работать существенно быстрее (раза в 3);
2. Небольшие модели перестали страдать от зашумленного контекста и стали точнее.

У меня было ещё несколько идей для экспериментов, однако подходило время дедлайна, и я решил остановиться на текущей версии.

4. Разбор полетов

Рефлексируя над процессом выполнения проекта, я понял, что совершал две ошибки, исправление которых в дальнейшем поможет мне работать намного эффективнее. В этой главе расскажу про них, а также про идеи улучшения системы, которые не успел реализовать.

Планирование и документация экспериментов

Та цепочка улучшений, которую я описал в этой статье -- лишь небольшая подборка моих лучших экспериментов. Почти все остальные были **плохо продуманы**, а их **результаты не зафиксированы**.

Почему так получилось?

Как вы помните, Эндрю в своем курсе советовал быстро тестировать гипотезы, чтобы не сидеть долго над созданием идеальной системы, которая не будет работать на практике. Я так вдохновился этим, что незаметно для себя **ушел в другую крайность** -- делал множество экспериментов, большая часть которых была не спланирована, а их логи захламляли Opik. Поэтому мне приходилось их удалять. Из-за этого я по ошибке (или чтобы вспомнить результаты) повторял уже проведенные когда-то эксперименты.

Таким образом, существуют две крайности:

1. Долго планировать идеальную систему, пытаясь реализовать все до первого запуска.
2. Разрабатывать без плана, совершая небольшие изменения и сразу же тестируя их.

А лучший подход, конечно, где-то между ними: создать бейзлайн и итеративно улучшать его, тщательно **документируя** и **планируя** отдельные эксперименты.

Звучит как что-то банальное, что рассказывали на первых курсах по программированию, но часто так и получается -- простые вещи по-настоящему осознаются только на практике (или я просто плохо ботал).

Как можно улучшить систему?

Если вы прочитали до этого места, то у вас наверняка появились свои собственные идеи по улучшению обсуждаемого агента. Более того, наверняка появились идеи совершенно новых архитектур. В этом сложность и прелесть инженерных задач -- **получить хорошее решение можно множеством разных способов**.

Поэтому я не хочу приводить тут кучу своих идей. Тем более заранее не понятно какие из них promising (как сказал бы chatgpt). Кроме одной, которая **почти наверняка поможет улучшить качество/робастность работы агента**.

Помните, я рассказывал про способы сообщить LLM о природе запросов и объектов, которые ей придется анализировать? Я выделял их два: примеры и инструкции. В конечном итоге оба используются: примеры подбираются динамически из базы знаний, а инструкции я прописывал ручками, обобщая ошибки модели. Однако тут есть засада.

Эти инструкции я прописывал, смотря на ошибки двух-трех конкретных моделей. Но ведь другие LLM совершенно не обязательно имеют такие же проблемы при оценке релевантности -- им нужны другие инструкции. Делать один большой список не вариант, все не учтешь (+контекст не резиновый). Так что делать?

Для каждой модели подбирать свои инструкции. Но не ручками, а с помощью другой LLM. Это можно реализовать самостоятельно либо с помощью фреймворка DSPy[5]. В этом вся его идеология -- не писать промпты вручную (ведь они зависят от моделей), а создавать их динамически, анализируя ошибки агента с помощью другой LLM.

В рамках проекта проверить эту фичу я не успел, но, когда появится подходящая задача или свободное время (ха-ха), обязательно посмотрю, как работает такой динамический промптинг.

Outro

Долго писал этот текст, переписывал и очень рад, что не бросил. Ведь слишком много полезного опыта было получено, и написание первой статьи на личном сайте позволило окончательно осознать его и зафиксировать. Конечно, многие технические детали я не осветил, и, если они вам интересны -- обязательно переходите на гитхаб!

Буду рад, если моя траектория выполнения проекта сможет стать неплохим обучающим примером для других людей или ИИ-агентов :)

О своих новых проектах на работе и учебе я тоже буду писать. Чтобы не пропустить, подписывайтесь на мой телеграм-канал!

А в конце хочу выразить благодарность ребятам из DLS[1], которые создают такую крутую бесплатную школу на русском языке!

Дополнения

Базовый системный промпт

```
def get_base_instructions():
    return """
You are a search relevance expert. Your goal is to assess if a map object satisfies a user query.

### DATA:

- SUMMARY (General Info): This is a broad overview of object. If it says "The shop sells electronics", it's a general truth.
- HIGHLIGHTS/DETAILS: These are specific snippets selected as most relevant to the query. They are NOT an exhaustive list.
- PRUNED NAMES: You only see names most similar to the query.

### VALID RELEVANCE VALUES:

- 1.0: Perfect match. The object definitely provides what the user wants.
- 0.0: Irrelevant. Wrong category, closed, or completely unrelated.
- 0.1: Partial/Unsure. The object might be relevant, but specific IMPORTANT constraints (price, specific service, rare item) are not exp
licitly confirmed in the provided snippets.

### INSTRUCTIONS:

1. Think step-by-step. Detect the USER INTENT: Is the user looking for a specific ITEM/SERVICE (e.g., "buy pills", "sauna") or a spe
cific VENUE TYPE (e.g., "Pharmacy", "Recreation Base")?

2. IF USER WANTS A VENUE TYPE (Category Search):
   - RUBRIC PRIORITY: The object's `Rubric` must semantically match the requested venue type.
   - MISMATCH PENALTY: If the user asks for "Category A" (e.g., "Holiday Center") and the object is "Category B" (e.g., "Sports Cam
p"), the relevance is likely 0.0, even if they share some features (like saunas or beds).
   - Exception: Only give 0.1 or 1.0 if Category B is relative (like "кафе" and "ресторан") or a direct sub-type or synonym of Category
A.

3. IF USER WANTS AN ITEM/SERVICE:
   - CATEGORY LOGIC: If the user asks for a COMMON item (e.g., "aspirin") and the object is a standard provider (Pharmacy), RELE
VANCE IS 1.0.
   - PARTIAL INVENTORY (The "Open List" Rule):
     - Treat 'Prices' and 'Description' as incomplete examples, not a full catalog.
     - If the object sells "Bags" but lists only "Wallets", assume it MIGHT sell "Suitcases".
     - DO NOT DOWNGRADE TO 0.0 just because a specific item or brand is missing from the text description, unless the category m
akes it impossible (e.g., asking for "Suitcase" in a "Bakery").
     - Verdict: If Category matches but Item/Brand is unconfirmed -> Relevance is 0.1.
   - Use 0.1 for RARE/SPECIFIC items where availability is truly unknown.

4. Pay attention to "Hard Constraints" (open now, free wifi) in user request.

5. QUERY RECOVERY (Typos & Layout Errors):
   - DETECT NOISE: Check if the query contains obvious typos (e.g., "купить машинист" instead of "купить машину") or wrong keyb
oard layout patterns (e.g., gibberish text that maps to meaningful words in another language/layout).
   - RECONSTRUCT INTENT: If the literal query is nonsensical but a highly probable correction exists, evaluate the object based on t
he CORRECTED query.
"""
```

Ссылки на материалы

1 Deep Learning School. [назад] [назад]

2 Andrew Ng. Agentic AI. [назад] [назад]

3 Comet. Opik. [назад]

4 Tavily. [назад]

5 DSPy. [назад]